

Teoria dos Grafos (COS 242) – Trabalho de Disciplina, Parte 1

Gabriel dos Santos Percu Saldanha (DRE 124007041) João Paulo Garcia de Assis Rangel (DRE 124130234)

Código-fonte da biblioteca e dos estudos de caso:

https://github.com/joaopaulogarciarangel/trabalho_pratico_grafos

1 Introdução

A biblioteca foi escrita em C++17 e compilada com CMake. Ela lê grafos não direcionados no formato do enunciado, representa o grafo por lista ou por matriz de adjacência (quem usa a biblioteca escolhe), faz BFS e DFS a partir de um vértice dado, calcula distâncias, componentes conexas, estatísticas de grau e diâmetro. O repositório traz, além da biblioteca, o programa de linha de comando **grafos**, que expõe essas funções e grava os resultados em arquivo, e o programa **estudos**, usado para gerar as tabelas da Seção 4.

2 Decisões de projeto

2.1 Representações

A **matriz de adjacência** é um bitset: um bloco contíguo de $n \cdot \lceil n/64 \rceil$ palavras de 64 bits, ou seja, um bit por par de vértices. Isso reduz a memória em 8 vezes em relação a uma matriz de `bool` e permite percorrer os vizinhos de um vértice palavra a palavra, pulando os zeros com as instruções de contagem de bits. O teste de adjacência é $O(1)$.

A **lista de adjacência** é um `vector<vector<int32_t>>`. Cada lista é reservada com o grau lido do arquivo, depois ordenada e sem repetições. Como fica ordenada, o teste de adjacência é feito por busca binária.

As duas classes têm a mesma interface, mas sem funções virtuais. A classe **Grafo** guarda um `std::variant` com uma delas e os algoritmos são templates sobre o tipo concreto. Com isso não há chamada virtual dentro dos laços de busca e a diferença de tempo medida vem só da estrutura de dados. Na interface pública os vértices vão de 1 a n ; internamente, de 0 a $n - 1$. Erros são sinalizados por exceção.

2.2 Leitura do arquivo

O arquivo é lido inteiro de uma vez (`fread`) e interpretado manualmente. Aceitamos `\r\n`, tabulações, espaços extras e linhas vazias; colunas a partir da terceira são ignoradas, já pensando nos pesos da Parte 2. Laços e arestas repetidas são descartados e contados, e m é o número de arestas distintas. Vértice fora de $1..n$ ou linha mal formada geram erro com o número da linha. Antes de construir a estrutura, os arrays temporários são liberados e a memória é devolvida ao sistema (`malloc_trim`), para que a medição de memória reflita só o grafo.

2.3 Buscas

Os vizinhos são sempre visitados em ordem crescente. Assim as árvores são determinísticas e saem iguais nas duas representações. A **BFS** marca o vértice ao descobri-lo e usa como fila um array de tamanho n com dois índices. A **DFS** é iterativa: a pilha guarda pares (vértice, pai), o vértice é marcado quando sai da pilha e os vizinhos entram em ordem decrescente, de modo que o menor sai primeiro. Os testes confirmam que ela gera a mesma árvore da versão recursiva, sem o risco de estourar a pilha de chamadas em grafos com milhões de vértices. Para cada vértice guardamos pai e nível (a raiz tem nível 0); o arquivo de saída traz, em cada linha, vértice, pai, nível e ordem de visita.

2.4 Estatísticas, componentes e distância

Grau mínimo, máximo e mediana saem de um histograma de graus, sem ordenar os n valores; o grau médio é $2m/n$. As componentes são encontradas com BFS a partir de cada vértice ainda sem rótulo, reaproveitando uma única fila para todas elas. O comando **estatisticas** do programa **grafos** grava tudo isso num arquivo texto: número de vértices e de arestas, graus mínimo, máximo e médio, mediana de grau, número de componentes e, para cada componente, o tamanho e a lista dos seus vértices, da maior componente para a menor. A distância entre dois vértices é uma BFS que para assim que o destino é descoberto; se eles estão em componentes diferentes, a distância é infinita.

2.5 Diâmetro

Em grafo desconexo, consideramos o maior diâmetro entre as componentes. A única primitiva usada é a BFS, conforme pedido. Implementamos três versões:

- **Ingênua**: uma BFS por vértice. Usada só nos testes, como referência.
- **Aproximada (4-sweep)**: em cada componente, BFS a partir do vértice de maior grau, depois a partir do

vértice mais distante encontrado, e assim por diante, passando pelo meio dos caminhos obtidos (5 BFS por componente). Como $\text{ecc}(x) \leq D \leq 2 \text{ecc}(x)$ para qualquer x , o resultado é um intervalo: o limite inferior é a maior excentricidade vista e o superior é o menor 2 ecc calculado.

- **Exata (iFUB, Crescenzi et al., 2013):** parte do vértice central u achado pelo 4-sweep e processa os vértices por camadas de distância a u , da mais distante para a mais próxima, calculando a excentricidade de cada um. Quando a maior excentricidade encontrada supera $2(i - 1)$, onde i é a camada atual, nenhum vértice mais interno pode superá-la e o valor é exato. Em grafos com poucos vértices nas camadas externas isso termina com muito menos que n BFS.

O algoritmo exato tem um limite de tempo (30 minutos por padrão). Se ele estoura, a função devolve o melhor intervalo [inferior, superior] obtido até ali, em vez de nada.

2.6 Testes e ferramentas

O projeto usa C++17, g++ e CMake. A versão de medição é compilada em Release com `-O2 -DNDEBUG`, sem `-march=native`, para que o binário rode em qualquer máquina x86-64. Os testes usam o doctest e cobrem os exemplos das aulas e do enunciado, a comparação entre lista e matriz (mesmas árvores, distâncias, componentes e estatísticas) e a comparação com implementações de referência escritas à parte, sem nenhuma otimização: DFS recursiva, BFS com `std::queue` e diâmetro ingênuo. Além dos casos fixos, rodamos essas comparações em milhares de grafos aleatórios. Há ainda uma configuração de depuração com AddressSanitizer e UBSan, que usamos para garantir que não havia acesso fora dos limites de memória nem comportamento indefinido.

2.7 Preparação para as próximas partes

A leitura já aceita a terceira coluna e a função de carga já recebe o parâmetro `direcionado`. Para grafos com pesos, a matriz precisará de outra classe (pesos não cabem num bitset), mas os algoritmos template continuam valendo para qualquer classe que implemente a mesma interface.

3 Metodologia dos estudos de caso

Rodamos tudo numa máquina com AMD Ryzen 5 5500, em Linux sob WSL2 com cerca de 7,7 GB de RAM disponíveis, compilando com g++ 15.2 em modo Release (`-O2`). Cada grafo foi processado em uma execução separada por representação, para que a memória de uma não interferisse na outra.

Memória: lemos o `VmRSS` do processo (`/proc/self/status`) antes e depois de carregar o grafo e registramos a diferença. Também registramos o tamanho teórico da estrutura, para comparação. Antes de carregar a matriz, o programa lê só o n e estima o espaço necessário; se passa de 80% da memória livre, a matriz não é alocada e o caso fica marcado como *inviável*.

Tempo: sorteamos 100 vértices distintos (gerador `mt19937_64`, semente 42) e usamos os mesmos como origem para BFS e DFS nas duas representações. Cada busca é cronometrada isoladamente com o relógio monotônico do C++ (`steady_clock`), sem incluir leitura do arquivo nem escrita de resultados. Uma BFS de aquecimento é descartada antes das medições.

4 Estudos de caso

4.1 Características dos grafos

Tabela 1: Estatísticas de grau de cada grafo e entradas do arquivo descartadas na leitura.

Grafo	Grau				Descartadas	
	mínimo	máximo	médio	mediana	laços	repetidas
1	8	43	21,98	22	4	52
2	1	112	52,00	55	65	1.019
3	1	15	4,08	4	1	2
4	9	89	43,66	47	35	406
5	1	22	5,44	5	7	10
6	2	56	19,19	20	43	148

Os grafos 1 e 2 são pequenos, os grafos 3 e 4 têm 375 mil vértices e os grafos 5 e 6 têm cerca de 4,8 milhões. Os grafos 3 e 4 têm a mesma quantidade e os mesmos tamanhos de componentes, assim como os grafos 5 e 6, mas o segundo de cada par tem bem mais arestas: o grau médio passa de 4,1 para 43,7 entre os grafos 3 e 4, e de 5,4 para 19,2 entre os grafos 5 e 6. Todos os arquivos tinham alguns laços e arestas repetidas, que foram ignorados.

4.2 Memória (pergunta 1)

Tabela 2: Memória usada pelo grafo carregado, em MB (1 MB = 1024^2 bytes). “Medida” é a variação do RSS; “estimada” é o tamanho teórico da estrutura.

Grafo	n	m	Lista		Matriz	
			medida	estimada	medida	estimada
1	10.000	109.921	1,2	1,1	12,0	12,0
2	49.948	1.298.710	11,9	11,2	297,8	297,8
3	375.000	765.615	21,9	15,9	inviável	16.767
4	375.000	8.186.986	77,5	72,5	inviável	16.767
5	4.843.750	13.168.911	303,4	229,8	inviável	2.796.912
6	4.843.750	46.469.479	549,4	483,9	inviável	2.796.912

A matriz ocupa $n^2/8$ bytes independentemente do número de arestas, enquanto a lista cresce com $n + 2m$. Nos grafos 1 e 2, onde as duas couberam na memória, a matriz gastou 10 e 25 vezes mais. Do grafo 3 em diante a matriz é impraticável: seriam cerca de 16 GB para os grafos 3 e 4 e cerca de 2,7 TB para os grafos 5 e 6, muito além dos 7,7 GB da máquina. Mesmo usando um bit por par, a matriz só é uma opção para grafos com dezenas de milhares de vértices. A diferença entre a memória medida e a estimada na lista (de 5% a 38%, maior nos grafos esparsos) vem do alocador, que reserva cada um dos milhões de vetores pequenos com algum espaço extra.

4.3 Tempo de BFS e DFS (perguntas 2 e 3)

Tabela 3: Tempo médio de uma busca, em milissegundos, sobre 100 origens distintas (média $\pm$ desvio padrão).

Grafo	Lista		Matriz	
	BFS	DFS	BFS	DFS
1	$0,39 \pm 0,02$	$1,43 \pm 0,08$	$3,51 \pm 0,17$	$4,81 \pm 0,16$
2	$2,32 \pm 1,66$	$4,15 \pm 2,85$	$23,20 \pm 14,91$	$20,48 \pm 13,25$
3	$16,19 \pm 6,07$	$46,48 \pm 14,23$	inviável	inviável
4	$74,03 \pm 22,86$	$112,16 \pm 43,60$	inviável	inviável
5	$302,29 \pm 152,86$	$799,96 \pm 423,37$	inviável	inviável
6	$977,18 \pm 650,95$	$1.215,62 \pm 723,34$	inviável	inviável

Nos grafos 1 e 2 a lista foi cerca de 10 vezes mais rápida na BFS e de 3 a 5 vezes mais rápida na DFS. Na lista, cada busca percorre só os vizinhos existentes, $O(n + m)$; na matriz, precisa varrer a linha inteira de cada vértice, $O(n^2/64)$ com o bitset. No grafo 1, por exemplo, isso dá cerca de 1,6 milhão de palavras lidas por busca contra 220 mil entradas na lista. A DFS foi mais lenta que a BFS na lista porque a pilha recebe um par para cada aresta que leva a um vértice ainda não explorado, enquanto a fila da BFS recebe cada vértice uma única vez. Na matriz essa diferença pesa menos, porque o custo de varrer as linhas domina.

O desvio padrão alto a partir do grafo 2 não é ruído de medição. Esses grafos têm várias componentes (Tabela 6) e, quando a origem sorteada cai numa componente pequena, a busca termina muito antes. No grafo 1, que é conexo, o desvio ficou abaixo de 6% da média.

4.4 Pais nas árvores de busca (pergunta 4)

Tabela 4: Pai dos vértices 10, 20 e 30 na árvore da BFS. “–” indica que o vértice não é alcançado a partir da raiz.

Grafo	Raiz 1			Raiz 2			Raiz 3		
	10	20	30	10	20	30	10	20	30
1	2.042	8.382	2.394	8.935	9.071	3.555	7.685	9.543	5.783
2	–	–	–	1.351	–	–	–	46.738	12.999
3	–	–	141.597	158.403	75.471	–	158.403	319.691	–
4	243.865	370.783	136.244	–	–	–	–	–	–
5	1.888.350	–	2.502.539	–	–	–	1.888.350	–	191.713
6	–	–	–	1.677.854	3.607.226	3.898.629	–	–	–

Tabela 5: Pai dos vértices 10, 20 e 30 na árvore da DFS.

Grafo	Raiz 1			Raiz 2			Raiz 3		
	10	20	30	10	20	30	10	20	30
1	709	666	86	709	666	86	709	666	86
2	–	–	–	3.946	–	–	–	217	3.513
3	–	–	141.597	192.218	141.526	–	106.718	141.526	–
4	12.269	10.738	1.531	–	–	–	–	–	–
5	1.888.350	–	191.713	–	–	–	1.888.350	–	2.502.539
6	–	–	–	381.031	431.008	446.011	–	–	–

Nos grafos 1 e 2 as árvores foram idênticas com lista e com matriz, o que era esperado, já que as duas visitam os vizinhos em ordem crescente. Os traços aparecem quando a raiz e o vértice estão em componentes diferentes; no grafo 2, por exemplo, o vértice 1 está numa componente que não contém 10, 20 nem 30. No grafo 1 a DFS deu os mesmos pais para as três raízes. Como a DFS segue sempre pelo menor vizinho e vai longe antes de voltar, é provável que as três execuções cheguem ao trecho do grafo que contém 10, 20 e 30 pelo mesmo caminho. Na BFS isso não acontece, porque o pai de um vértice depende da sua distância até a raiz.

4.5 Distâncias e componentes conexas (perguntas 5 e 6)

Tabela 6: Distâncias entre os pares pedidos e componentes conexas. ∞ indica vértices em componentes diferentes.

Grafo	Distância			Componentes		
	(10,20)	(10,30)	(20,30)	quantidade	maior	menor
1	3	3	4	1	10.000	10.000
2	∞	∞	3	10	25.000	48
3	9	∞	∞	2	250.000	125.000
4	4	3	4	2	250.000	125.000
5	∞	9	∞	5	2.500.000	156.250
6	5	5	5	5	2.500.000	156.250

As distâncias batem com as Tabelas 4 e 5: sempre que uma raiz alcança um vértice do par e não o outro, os dois estão em componentes diferentes e a distância é infinita. Apesar de terem componentes com os mesmos tamanhos, os grafos 3 e 4 não distribuem os vértices da mesma forma: no grafo 3 o vértice 30 fica numa componente diferente de 10 e 20, e no grafo 4 os três estão juntos. O mesmo vale para os grafos 5 e 6.

4.6 Diâmetro (pergunta 7)

Tabela 7: Diâmetro calculado com a representação por lista. Intervalos [inferior, superior] indicam que o valor exato não foi determinado. “BFS” é o número de buscas feitas.

Grafo	Aproximado (4-sweep)			Exato (iFUB)		
	valor	tempo	BFS	valor	tempo	BFS
1	[4, 8]	2,3 ms	5	5	3,7 s	9.489
2	[20, 24]	34,5 ms	50	20	34,9 ms	57
3	[21, 30]	173,6 ms	10	22	229,5 s	25.744
4	[5, 10]	725,3 ms	10	[5, 10]	30 min (limite)	20.794
5	[58, 78]	5,1 s	25	59	77,1 s	5.181
6	[19, 28]	12,5 s	25	[19, 20]	30 min (limite)	95.545

O 4-sweep roda em milissegundos ou poucos segundos e, nos quatro grafos em que o exato terminou, o limite inferior ficou a no máximo uma unidade do diâmetro real. Para grafos muito grandes ele é uma boa estimativa. O número de BFS do aproximado é sempre 5 por componente (no grafo 2, 10 componentes dão 50). Já o iFUB variou muito: no grafo 2 bastaram 57 BFS, só 7 a mais que o aproximado, e no grafo 5 foram 5.181, contra os 4,8 milhões que o método ingênuo faria. No grafo 1 foram 9.489 BFS para 10.000 vértices, quase o mesmo que o ingênuo, porque o grafo tem diâmetro pequeno e quase todos os vértices ficam nas camadas externas.

Nos grafos 4 e 6 o limite de 30 minutos foi atingido. O grafo 4 é o caso mais desfavorável para o iFUB: é denso (grau médio 43,7, Tabela 1) e tem diâmetro pequeno, então a camada mais distante do centro concentra muitos vértices. O limite superior só diminui quando uma camada inteira é processada, e após 20.794 BFS a primeira camada ainda não tinha terminado; por isso o intervalo continuou [5, 10]. No grafo 6 o algoritmo avançou mais e o diâmetro ficou entre 19 e 20. Para o grafo 4, a melhor estimativa que temos é o limite inferior: nos quatro grafos resolvidos, ele ficou a no máximo uma unidade do valor exato, o que sugere um diâmetro de 5 ou 6.

5 Conclusão

A lista de adjacência foi melhor nos dois critérios pedidos: gastou de 10 a 25 vezes menos memória e fez as buscas de 3 a 10 vezes mais rápido nos grafos em que a comparação foi possível, e foi a única que coube na memória para os quatro grafos maiores. A matriz só compensaria em grafos pequenos e densos, onde o teste de adjacência em $O(1)$ tivesse papel importante, o que não foi o caso destes estudos. Para o diâmetro, a combinação de 4-sweep e iFUB resolveu quatro dos seis grafos de forma exata e deixou o grafo 6 com uma unidade de incerteza, algo que a abordagem ingênua não conseguiria nos grafos com milhões de vértices. O grafo 4, denso e de diâmetro pequeno, ficou como o caso em aberto.